Machine Learning Practical No. 2

Dimensionality Reduction using PCA and LDA

Aim: Apply dimensionality reduction using PCA and LDA on the Diabetes dataset and observe the influence of feature reduction on model accuracy and computational efficiency.

Theory / Key Concepts

Dimensionality Reduction: Reducing the number of features while retaining useful information.

PCA: Principal Component Analysis transforms original features into a smaller set of uncorrelated principal components. PCA does not use class labels to find components.

LDA: Linear Discriminant Analysis uses class labels and finds directions that best separate the classes.

StandardScaler: Standardizes numerical features to approximately mean 0 and standard deviation 1.

Logistic Regression: A classification algorithm used here to compare model performance before and after dimensionality reduction.

Step-by-Step Procedure

Step 1 — Open Anaconda Navigator: Start Menu → Anaconda Navigator.

Step 2 — Launch Jupyter Notebook: Click Launch under Jupyter Notebook.

Step 3 — Create Notebook: New → Python 3. Use a suitable name such as PCA_LDA.ipynb.

Step 4 — Import Libraries

```
import time
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
```

Step 5 — Load Dataset and Separate X/y

```
data = pd.read_csv("diabetes.csv")
X = data.drop("Outcome", axis=1)
y = data["Outcome"]
print("Dataset Shape:", X.shape)
```

X contains the input features and **y** contains the target column Outcome.

Step 6 — Split Dataset

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)
```

Step 7 — Feature Scaling

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

The scaler learns mean and standard deviation from training data and applies the same scaling to the test data.

Step 8 — Original Model

```
model = LogisticRegression(max_iter=1000)
start = time.time()
model.fit(X_train_scaled, y_train)
train_original = time.time() - start

start = time.time()
```

```
pred_original = model.predict(X_test_scaled)
test_original = time.time() - start

acc_original = accuracy_score(y_test, pred_original)
```

Step 9 — PCA

```
pca = PCA(n_components=5)
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

model_pca = LogisticRegression(max_iter=1000)
start = time.time()
model_pca.fit(X_train_pca, y_train)
train_pca = time.time() - start

start = time.time()
pred_pca = model_pca.predict(X_test_pca)
test_pca = time.time() - start

acc_pca = accuracy_score(y_test, pred_pca)
```

PCA reduces the original 8 features to 5 principal components.

Step 10 — LDA

```
lda = LinearDiscriminantAnalysis(n_components=1)
X_train_lda = lda.fit_transform(X_train_scaled, y_train)
X_test_lda = lda.transform(X_test_scaled)

model_lda = LogisticRegression(max_iter=1000)
start = time.time()
model_lda.fit(X_train_lda, y_train)
train_lda = time.time() - start

start = time.time()
pred_lda = model_lda.predict(X_test_lda)
test_lda = time.time() - start

acc_lda = accuracy_score(y_test, pred_lda)
```

For a binary classification problem, LDA can reduce the data to one discriminant component.

Step 11 — Compare Results

```
results = pd.DataFrame({
    "Method": ["Original", "PCA", "LDA"],
    "Features": [8, 5, 1],
    "Accuracy": [
        round(acc_original * 100, 2),
        round(acc_pca * 100, 2),
        round(acc_lda * 100, 2)
    ],
    "Training Time (sec)": [
        round(train_original, 6),
        round(train_pca, 6),
        round(train_lda, 6)
    ],
    "Prediction Time (sec)": [
        round(test_original, 6),
        round(test_pca, 6),
        round(test_lda, 6)
    ]
})
print(results)
```

Complete Program

```
import time
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Load Dataset
data = pd.read_csv("diabetes.csv")
X = data.drop("Outcome", axis=1)
y = data["Outcome"]
print("Dataset Shape:", X.shape)

# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)

# Feature Scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Original
model = LogisticRegression(max_iter=1000)
start = time.time()
model.fit(X_train_scaled, y_train)
train_original = time.time() - start

start = time.time()
pred_original = model.predict(X_test_scaled)
test_original = time.time() - start
acc_original = accuracy_score(y_test, pred_original)

# PCA
pca = PCA(n_components=5)
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

model_pca = LogisticRegression(max_iter=1000)
start = time.time()
model_pca.fit(X_train_pca, y_train)
train_pca = time.time() - start

start = time.time()
pred_pca = model_pca.predict(X_test_pca)
test_pca = time.time() - start
acc_pca = accuracy_score(y_test, pred_pca)

# LDA
lda = LinearDiscriminantAnalysis(n_components=1)
X_train_lda = lda.fit_transform(X_train_scaled, y_train)
X_test_lda = lda.transform(X_test_scaled)

model_lda = LogisticRegression(max_iter=1000)
start = time.time()
model_lda.fit(X_train_lda, y_train)
train_lda = time.time() - start

start = time.time()
pred_lda = model_lda.predict(X_test_lda)
test_lda = time.time() - start
acc_lda = accuracy_score(y_test, pred_lda)

# Results
results = pd.DataFrame({
    "Method": ["Original", "PCA", "LDA"],
    "Features": [8, 5, 1],
    "Accuracy": [
        round(acc_original * 100, 2),
        round(acc_pca * 100, 2),
        round(acc_lda * 100, 2)
    ],
    "Training Time (sec)": [
        round(train_original, 6),
        round(train_pca, 6),
```

```
            round(train_lda, 6)
        ],
        "Prediction Time (sec)": [
            round(test_original, 6),
            round(test_pca, 6),
            round(test_lda, 6)
        ]
    })
    print(results)
```

Quick Viva Revision

What is dimensionality reduction? Reducing the number of features while retaining useful information.

What is PCA? An unsupervised dimensionality-reduction technique that creates principal components.

What is LDA? A supervised technique that uses class labels to find directions that separate classes.

Why StandardScaler? To put features on a comparable scale before PCA/LDA.

Why Logistic Regression? It provides a simple classification model for comparing accuracy before and after reduction.

Why PCA n_components=5? To reduce 8 original features to 5 components in this practical.

Why LDA n_components=1? With two classes, one discriminant component is sufficient.

What does time.time() measure? Training and prediction time.

What does accuracy_score do? It calculates the proportion of correct predictions.

Exam Memory Trick

SCALE → **ORIGINAL** → **PCA** → **LDA** → **COMPARE**

Original = 8 features → PCA = 5 features → LDA = 1 feature → compare Accuracy + Training Time + Prediction Time.